

## **SIMATS ENGINEERING**

### **ASSESSMENT TOOL 1**

**COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02**

#### **COURSE OUTCOME COVERED**

**CO5: Apply computer vision techniques to real-world applications such as face recognition, tracking, medical imaging, and autonomous systems. (BL3)**

**Assessment Tool Weightage: 50%**

**QUESTIONS: 5**

**TotalMarks:50**

#### **Annexure A**

#### **Constraint-Based Problem Solving**

##### **Code of Conduct:**

**I B.Deepthi-192424269 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.**

**I understand that any violation of academic integrity rules will result in disciplinary action.**

**Signature of the student:B.Deepthi**

# Question 1: Secure Face Recognition in a Low-Light Environment

## Scenario

A company wants to deploy a secure face-recognition system on an embedded device. The system must work in low-light conditions, recognize faces with orientations up to  $\pm 30^\circ$ , and produce results in less than 200 ms per face.

## 1. Adapting Eigenface Recognition for Illumination and Orientation

### Problem Understanding

The conventional Eigenface method is sensitive to changes in lighting and facial pose. Therefore, directly applying Eigenfaces to low-light images may produce poor recognition accuracy.

### Constraint Analysis

The important constraints are:

- **Low illumination:** Facial features may become unclear.
- **Pose variation:** The face can rotate up to  $\pm 30^\circ$ .
- **Limited hardware:** The embedded device has restricted processing power.
- **Real-time requirement:** Recognition must take less than 200 ms.

### Proposed Solution

I would modify the Eigenface pipeline as follows:

**Input Image → Face Detection → Illumination Normalization → Alignment → Feature Extraction → Eigenface Matching → Identity**

1. Detect the face using a lightweight detector.
2. Locate important landmarks such as eyes and nose.
3. Align the face using the detected landmarks.
4. Normalize illumination using histogram equalization or CLAHE.
5. Resize the aligned face to a fixed size.
6. Project the normalized face onto the Eigenface space.

7. Compare the resulting feature vector with stored face templates.

For  $\pm 30^\circ$  rotation, multiple reference images of the same person can be stored at different orientations. Alternatively, small pose normalization can be performed before Eigenface extraction.

## **Application of Concepts**

The solution applies:

- Eigenfaces/PCA for dimensionality reduction.
- Face alignment for pose normalization.
- Histogram equalization/CLAHE for illumination correction.
- Lightweight face detection for embedded deployment

## **2. Preprocessing for Low-Light Recognition**

A suitable preprocessing pipeline is:

**Camera Image → Denoising → Brightness Correction → Contrast Enhancement → Face Alignment → Normalization**

### **Step 1: Noise Reduction**

Low-light cameras often produce noise

### **Step 2: Gamma Correction**

Gamma correction can brighten dark facial regions without simply increasing all pixel values equally.

### **Step 3: CLAHE**

Contrast Limited Adaptive Histogram Equalization improves local contrast and is particularly useful when different parts of the face have different illumination.

### **Step 4: Face Alignment**

Eyes and other landmarks are used to align the face to a standard position.

### **Step 5: Normalization**

The image is converted to grayscale and normalized to a common intensity range before Eigenface extraction.

## **Why This Works**

The objective is not simply to make the image brighter. The objective is to make facial features such as the eyes, nose, and mouth more distinguishable while avoiding excessive noise amplification.

### 3. Optimizing Computation for Real-Time Performance

To satisfy the **<200 ms per-face constraint**, the following optimizations can be used:

1. **Reduce image resolution** before recognition.
2. Use a **lightweight face detector**.
3. Convert images to grayscale, and Perform Eigenface projection using a reduced number of principal components.
4. Store precomputed face templates.
5. Avoid repeatedly training PCA during recognition.
6. Use **ROI processing** so that only the detected face is processed.
7. Use efficient matrix operations.
8. Process frames selectively instead of processing every camera frame.
9. Use hardware acceleration where available.

#### Priority of Constraints

| Constraint          | Priority  | Solution                            |
|---------------------|-----------|-------------------------------------|
| Recognition time    | Very High | ROI + reduced PCA components        |
| Low light           | High      | CLAHE + gamma correction            |
| Pose variation      | High      | Alignment + multiple pose templates |
| Hardware limitation | Very High | Lightweight algorithms              |

#### Final Justification

The proposed system balances accuracy and speed rather than using a computationally expensive model. Preprocessing handles difficult illumination, alignment handles pose variation, and dimensionality reduction keeps Eigenface recognition suitable for embedded hardware

## Question 2: AI-Based Photo Album Organization

### Scenario

The system must organize millions of personal photographs according to people and events. Images may have different resolutions and poses, and privacy requires all processing to happen locally.

### 1. Grouping Photos of the Same Individual

#### Problem Understanding

The major task is to determine which photographs contain the same person and then organize those photographs into meaningful groups.

#### Constraint Analysis

The system must handle:

- Millions of images.
- Different image resolutions.
- Different facial poses.
- Multiple photographs of the same person.
- Local-only processing for privacy.

#### Proposed Solution

A practical pipeline is:

**Photos → Face Detection → Face Alignment → Face Embedding → Similarity Search → Clustering → Person Groups**

1. Detect all faces in the photographs.
2. Crop and align each face.
3. Convert each face into a numerical feature vector called a **face embedding**.
4. Compare embeddings using a distance measure such as cosine distance.
5. Group similar embeddings using clustering.
6. Assign a temporary identity such as Person A, Person B, etc.
7. Allow the user to provide names for the clusters.

For a large collection, an efficient approximate nearest-neighbor index can be used instead of comparing every face with every other face.

## **Application of Concepts**

This combines:

- Face detection.
- Face recognition.
- Feature extraction.
- Clustering.
- Similarity measurement.

## **Justification**

Face embeddings are more robust to changes in pose and image quality than direct pixel comparison. Clustering then allows the system to discover groups without requiring a name for every person in advance.

## **2. Detecting Occluded or Partial Faces**

A face may be partially hidden by:

- Sunglasses , Masks.
- Hair, Objects.
- Another person.
- Image cropping.

## **Proposed Method**

The system can calculate a **face-quality score** using:

- Face size.
- Percentage of visible facial area.
- Landmark confidence.
- Occlusion indicators.
- Embedding confidence.

Images with very low-quality or heavily occluded faces can be placed into a separate “**Needs Review**” group.

### **Decision Logic**

**Good face → Normal recognition**

**Partially visible face → Recognition with stricter threshold**

**Severely occluded face → Separate/review group**

This prevents unreliable matches from contaminating the main person clusters.

### **3. Accuracy vs Computational Efficiency**

There is an important trade-off.

| <b>Approach</b>               | <b>Accuracy</b> | <b>Computation</b> | <b>Suitable for</b>   |
|-------------------------------|-----------------|--------------------|-----------------------|
| High-resolution deep model    | Very high       | High               | Small collections     |
| Lightweight model             | Good            | Low                | Local devices         |
| Classical face features       | Moderate        | Low                | Very limited hardware |
| Approximate similarity search | Good            | Low for large data | Millions of images    |

### **Local Processing Strategy**

Since privacy is mandatory, the system should not upload photographs to a cloud server.

A better architecture is:

**Local Face Detection → Local Embedding Generation → Local Database → Local Clustering**

Only the final organization information is shown to the user.

### **Final Justification**

For millions of photographs, the best solution is not necessarily the most accurate model. A lightweight embedding model combined with efficient indexing and clustering provides a practical balance between accuracy, speed, storage, and privacy.

# **Question 3: Multi-Camera Pedestrian Detection and Tracking**

## **Scenario**

A smart-city surveillance system must detect and track pedestrians across multiple cameras. The environment contains frequent occlusions, network bandwidth is limited, and processing must be real time.

## **1. Foreground–Background Separation in Crowded Scenes**

### **Problem Understanding**

The objective is to identify moving pedestrians while avoiding false detections caused by shadows, illumination changes, camera noise, and other moving objects.

### **Constraint Analysis**

The main challenges are:

- Crowded scenes, Shadows.
- Frequent pedestrian occlusion.
- Changing illumination.
- Real-time processing.
- Limited bandwidth.

### **Proposed Algorithm**

A robust foreground-background pipeline can be:

**Video Frame → Background Model → Foreground Mask → Shadow Removal → Morphological Filtering → Object Detection**

#### **Step 1: Background Modeling**

Maintain a continuously updated background model using a method such as MOG2.

#### **Step 2: Foreground Extraction**

Subtract the current frame from the estimated background.

#### **Step 3: Shadow Suppression**

Use shadow detection or color/intensity information to remove regions that move like shadows but are not people.

#### **Step 4: Morphological Processing**

Apply opening to remove small noise and closing to fill gaps.

#### **Step 5: Object Filtering**

Remove objects that are too small or have unrealistic shapes.

#### **Step 6: Person Detection**

Use a lightweight pedestrian detector to verify candidate regions.

#### **Why It Reduces False Positives**

Background subtraction alone can classify shadows and illumination changes as moving objects. Combining background modeling with shadow suppression and person verification makes the final detection more reliable.

### **2. Particle Filters for Occlusion Handling**

A particle filter represents the possible state of a pedestrian using multiple particles.

Each particle represents a possible state:

**State = [x-position, y-position, velocity, scale]**

#### **Working**

1. Initialize particles around the detected pedestrian.
2. Predict the pedestrian's next position.
3. Compare predicted positions with the available image evidence.
4. Assign higher weights to particles that agree with the observation.
5. Resample the particles.
6. Estimate the pedestrian's current position.

#### **During Occlusion**

**Before Occlusion → Prediction → Occlusion → Continued Tracking → Reappearance → Correction**

#### **Advantage**

Particle filters can represent multiple possible locations, which is useful when the exact pedestrian position is uncertain.

### **3. Efficient Multi-Camera Identification**

A centralized system that continuously sends complete video streams would consume large amounts of bandwidth.

A better architecture is:

**Camera → Local Detection/Tracking → Compact Identity Features → Central Server**

Each camera performs initial processing locally.

Instead of transmitting full video continuously, it sends:

- Person bounding box.
- Timestamp.
- Camera ID, Tracking ID.
- Compact appearance/identity feature.

### **Cross-Camera Matching**

When a pedestrian disappears from Camera A and appears in Camera B, the system compares:

- Appearance features, Estimated walking direction.
- Time difference, Camera location.
- Bounding-box information.

A matching score can be calculated:

**Match Score = Appearance Similarity + Spatial Consistency + Temporal Consistency**

### **Final Justification**

Edge processing reduces network traffic, while feature-based matching provides consistent pedestrian identification. Particle filtering further improves tracking when pedestrians temporarily disappear.

## **Question 4: Autonomous Driving in Urban Streets**

### **Scenario**

A self-driving vehicle must detect roads, traffic signs, and pedestrians under rain, nighttime conditions, and dense traffic. The complete decision process must operate within **50 ms per frame**.

### **1. Road Detection Under Shadows, Rain, and Night**

#### **Problem Understanding**

Road detection is difficult because road appearance changes significantly with weather and lighting.

#### **Proposed Pipeline**

**Camera → Image Enhancement → Color/Feature Extraction → Road Segmentation → Morphological Filtering → Lane/Road Boundary Detection**

#### **Daytime**

Use color and edge information to identify road regions.

#### **Shadows**

Shadow-sensitive color spaces can help distinguish dark shadows from actual road boundaries.

#### **Rain**

Use denoising and contrast enhancement before road detection.

#### **Night**

Brightness enhancement and exposure normalization can improve visibility.

#### **Adaptive Processing**

Instead of using one fixed threshold, thresholds can be adapted according to image brightness and environmental conditions.

#### **Application of Concepts**

Useful OpenCV operations include:

- Gaussian filtering.

- Histogram/CLAHE enhancement.
- Color-space conversion.
- Edge detection, Thresholding.
- Morphological operations, Contour analysis.

### **Justification**

An adaptive pipeline is more reliable than a fixed threshold because road appearance is not constant across weather and lighting conditions.

## **2. Efficient Detection of Road Signs and Pedestrians**

A parallel processing pipeline can be used:

### **Camera Frame**

→ **Preprocessing**

→ **Branch 1: Road/Lane Detection**

→ **Branch 2: Traffic Sign Detection**

→ **Branch 3: Pedestrian Detection**

→ **Object Fusion**

→ **Decision Module**

### **Road/Lane Detection**

Use edge detection and geometric features or a lightweight segmentation method.

### **Traffic Signs**

Use a lightweight object detector trained for traffic-sign categories.

### **Pedestrians**

Use a lightweight pedestrian detector.

### **Efficiency Techniques**

- Resize input frames appropriately.
- Process only the required region of interest.
- Use lightweight detection models.

- Run expensive detection periodically while tracking objects between detections.

## Final Output

The system produces:

**Road/Lane Information + Sign Information + Pedestrian Locations**

which are passed to the vehicle's decision-making system.

## 3. Deep Learning vs Classical Computer Vision

| Feature                    | Deep Learning              | Classical Computer Vision |
|----------------------------|----------------------------|---------------------------|
| Accuracy in complex scenes | Generally high             | Moderate                  |
| Training requirement       | Requires dataset/training  | Usually no training       |
| Computation                | Usually high               | Usually lower             |
| Adaptability               | Good after training        | Limited                   |
| Weather variation          | Can learn complex patterns | More sensitive            |
| Embedded deployment        | Requires optimization      | Easier                    |
| Development complexity     | Higher                     | Lower                     |

### Trade-Off

Deep learning is usually better for recognizing complex objects such as pedestrians and traffic signs. However, it can require significant computational resources.

Classical techniques such as edges, contours, thresholding, and morphology are computationally cheaper and can be useful for road/lane geometry.

### Recommended Hybrid Approach

**Classical CV → Road/Lane Detection**

**Lightweight Deep Learning → Pedestrians and Traffic Signs**

**Tracking → Reduce repeated detection**

### **Final Justification**

A hybrid architecture is preferable under a 50 ms constraint because it uses classical methods where they are computationally efficient and deep learning where semantic understanding is important.

## **Question 5: Real-Time AR Surgical Assistance**

### **Scenario**

An augmented-reality surgical system must segment organs from high-resolution CT/MRI data and overlay them accurately in real time. Latency must remain below 100 ms and the wearable AR headset has limited GPU memory.

### **1. Real-Time Organ Segmentation**

#### **Problem Understanding**

Medical images are large and contain detailed anatomical information. A full-resolution segmentation model may exceed the memory and latency limitations of a wearable device.

#### **Constraint Analysis**

The major constraints are:

- High-resolution CT/MRI data.
- Less than 100 ms latency.
- Limited GPU memory.
- High segmentation accuracy.
- Accurate real-time overlay.

#### **Proposed Architecture**

**CT/MRI → Preprocessing → Region of Interest → Lightweight Segmentation Network → Postprocessing → 3D Reconstruction → AR Overlay**

#### **Step 1: Preprocessing**

Normalize intensity values and remove irrelevant image regions.

### **Step 2: ROI Extraction**

Instead of processing the complete volume at maximum resolution, identify the anatomical region relevant to the operation.

### **Step 3: Lightweight Segmentation**

Use a compact segmentation architecture or a reduced version of a U-Net-type model.

### **Step 4: Patch-Based Processing**

Large volumes can be divided into smaller patches to reduce memory usage.

### **Step 5: Postprocessing**

Remove small false regions using morphological operations or connected-component filtering.

### **Step 6: Output**

Generate a 3D organ mask that can be transformed into the AR coordinate system.

### **Justification**

ROI and patch-based processing significantly reduce the amount of data processed at one time, helping satisfy the memory and latency constraints.

## **2. Accurate AR/MR Overlay**

Accurate overlay requires alignment between the medical-image coordinate system and the physical patient.

### **Registration Pipeline**

**Medical Image → 3D Organ Model**

•

**Camera/AR Sensor → Patient Surface**

↓

**Coordinate Registration**

↓

**Transformation**



## AR Overlay

### Important Techniques

1. **Image-to-patient registration:** Align the CT/MRI model with the actual patient.
2. **Pose estimation:** Continuously estimate the headset/camera position.
3. **Coordinate transformation:** Convert organ coordinates into the AR coordinate system.
4. **Depth information:** Use depth sensing where available.
5. **Continuous tracking:** Update the overlay when the surgeon or patient moves.

### Accuracy Requirement

Even a small registration error can cause the virtual organ boundary to appear at the wrong physical location. Therefore, registration and tracking are as important as segmentation accuracy.

### Final Justification

A highly accurate segmentation model is not sufficient by itself. The segmented organ must also be geometrically registered to the patient for a clinically meaningful AR overlay.

### 3. Reducing Computational Load While Maintaining Accuracy

Several optimization strategies can be combined.

#### 1. ROI Processing

Process only the anatomical region needed for the current procedure.

#### 2. Model Compression

Reduce network parameters through pruning or other compression techniques.

#### 3. Quantization

Use lower-precision numerical representations where accuracy remains acceptable.

#### 4. Knowledge Distillation

Train a smaller student model to reproduce the behavior of a larger teacher model.

## 5. Patch-Based Processing

Process smaller portions of the medical volume instead of the complete high-resolution volume.

## 6. Frame/Update Skipping

The segmentation need not necessarily be recomputed from scratch for every AR update. Tracking can be used between expensive segmentation updates.

## 7. GPU Memory Optimization

Reuse memory buffers and avoid unnecessary copies of high-resolution data.

## 8. Multi-Resolution Processing

Use lower resolution for coarse localization and higher resolution only around important boundaries.

## Constraint Priority

| Constraint            | Priority  | Proposed Solution             |
|-----------------------|-----------|-------------------------------|
| Latency <100 ms       | Very High | Lightweight model + ROI       |
| Limited GPU memory    | Very High | Patches + compression         |
| Segmentation accuracy | Very High | Multi-resolution processing   |
| Overlay accuracy      | Very High | Registration + tracking       |
| High-resolution data  | High      | ROI + hierarchical processing |

## Final Justification

The best solution is not to reduce image quality blindly. Instead, computation should be concentrated on clinically important regions. Model compression, ROI extraction, patch processing, and efficient registration allow the system to maintain useful segmentation accuracy while meeting real-time requirements.